

Objects You Already Use

COMSC-1033 Computer Science I

Jeremy Evert

SWOSU

Week 9

Let's begin with values already present in ordinary Python code: a name, a message, or a list of items.

Let's begin with values already present in ordinary Python code: a name, a message, or a list of items. Each one is a value in a Python program. Now widen the question. Do not ask only what the value is called. Ask what kind of thing it is, what information it currently holds, and what it knows how to do. That three-part question is our object model. A value has a type. Its current text or items are its state. Its type supplies behaviors, including methods. A string is therefore more than characters sitting in a variable. A list is more than several values between brackets. Both are objects that Python can work with according to their types. This is not yet a lesson about writing your own class. It is a way to look at values you already use. The name on the left is a reference we use to reach an object; the name itself is not the type. Two different strings can have different text and still have the same type. As you inspect each value, keep asking: what kind of object is this, what state does it hold, and what behavior can it offer?

Follow the dot

```
text = "python"
upper_text = text.upper()
print(upper_text)  # expect: PYTHON
print(text)        # expect: python
print(type(upper_text).__name__)  # expect: str
pieces = text.split("h")
print(pieces)      # expect: ['pyt', 'on']
print(type(pieces).__name__)  # expect: list
```

The next clue is a dot.

The next clue is a dot. When you see a value followed by a dot and a name, Python is looking up something attached to that object. When the name identifies a method, parentheses call that method. In spoken language, you can hear it as: ask this object to use this behavior. A string can be asked to become uppercase, remove outside spaces, or replace one piece of text. The parentheses matter because they perform the call; without them, you are referring to the method itself rather than asking it to run. Not every name after a dot is a method. Some are attributes that hold information. Today we are concentrating on methods, because they make behavior visible. The dot is not decoration and it is not a universal command. It is a route from an object to a named part of what that object offers. This is why methods feel as though they belong to the string: the string's type supplies them. Pause and translate a line like "message dot upper with parentheses" into ordinary language: ask the message object for its uppercase behavior and call it. That translation will help you read programs before you can write every line yourself.

A string method gives back a value

```
text = "python"
upper_text = text.upper()
print(upper_text)  # expect: PYTHON
print(text)        # expect: python
print(type(upper_text).__name__)  # expect: str
pieces = text.split("h")
print(pieces)      # expect: ['pyt', 'on']
print(type(pieces).__name__)  # expect: list
```

Strings give us a useful surprise. A string method such as uppercase does not edit the original string in place.

Strings give us a useful surprise. A string method such as uppercase does not edit the original string in place. Strings are immutable, which means their contents cannot be changed in place. The method returns a string; the original is unchanged. CPython may hand back the same object when nothing changed, so compare values, not identity. If you call the method and throw away its result, the original reference still leads to the original text. If you assign the result back to the same variable, that variable now refers to the returned string. The important habit is to ask what came back. Some string methods return a string. The split method returns a list of pieces. The find method returns an integer position, while startswith returns a Boolean. A method call is not automatically an instruction that changes the object you started with. It may be a computation that produces a value. This distinction explains a common beginner error: saying that an uppercase call changes the string, then being surprised when printing the string shows the old text. Nothing mysterious happened. The call produced a result, and no one saved it. If you do save it, you have not repaired the old string; you have made your variable refer to the returned string. Think of a string method as sending a request to a text object and receiving a result in reply, then check the result's type instead of guessing from the receiver's type.

A list can change in place

```
route = ["red", "green"]
returned = route.append("blue")
print(route)  # expect: ['red', 'green', 'blue']
print(returned)  # expect: None
backup = route.copy()
print(backup)  # expect: ['red', 'green', 'blue']
print(type(backup).__name__)  # expect: list
```

Now compare a list. Lists are also objects, but a list is a mutable sequence. Mutable means the list can be changed in place.

Now compare a list. Lists are also objects, but a list is a mutable sequence. Mutable means the list can be changed in place. A method such as `append` adds an item to the existing list.

The list after the call contains the new item. The return value is a different question: `append` returns `None`. That is why the useful result is usually the changed list, not a new list stored

from `append`. The same pattern appears with other list methods that add, remove, or rearrange items. They act on the list itself. This is a place where type matters. The string method

`upper` returns a string; the original is unchanged because strings are immutable. The list method `append` changes a mutable list. Neither behavior is the definition of all methods. It is a

property you check for the particular type and method. A safe routine is: predict the object's state before the call, predict whether the call returns a value, run it, and inspect both. If you

assign the result of `append` to a variable, that variable holds `None` while the original list has changed. The list and the returned value are two separate things to keep track of.

Use the return value as evidence

```
text = "python"
upper_text = text.upper()
print(upper_text) # expect: PYTHON
print(text) # expect: python
print(type(upper_text).__name__) # expect:
    str
pieces = text.split("h")
print(pieces) # expect: ['pyt', 'on']
print(type(pieces).__name__) # expect: list
```

```
route = ["red", "green"]
returned = route.append("blue")
print(route) # expect: ['red', 'green', '
    blue']
print(returned) # expect: None
backup = route.copy()
print(backup) # expect: ['red', 'green', '
    blue']
print(type(backup).__name__) # expect: list
```

Let's make the contrast concrete. With a string, call the method and print both the returned value and the original. You will see two different strings: the new uppercase text and the old lowercase text. With a list, call append, keep the returned value, and then print both. You will see None in the saved result and the added item in the original list. That pair of observations is more useful than memorizing a slogan such as strings return and lists mutate. It teaches a question: what does this method return, and does it change its receiver? The word receiver simply means the object before the dot, the object receiving the request. The answer comes from the type's rules and the method's documentation. A string can also return a list from split, so even a method on an immutable object may return a different type. A list can have methods such as copy that return a list, and it also has methods that change the list. Types set the menu of behaviors, but individual methods determine the details. Use the examples as tiny experiments. Predict first. Run second. Explain the printed lines in plain language. That loop turns an accidental result into object knowledge.

Debug the missing result

Here is a small live debugging session. I start with `label = " python "` and write `'clean = label.strip()'`.

Here is a small live debugging session. I start with `label = " python "` and write `'clean = label.strip()'`. Then I write `'clean.upper()'` and print `'clean'`. The output is still `'python'`, in lowercase. My first thought might be, "upper did not work," but the evidence says something more precise: the call produced a string result, and I did not save it. I also used the parentheses correctly, so this is not the missing-parentheses bug. The repair is `'clean = clean.upper()'`, followed by `'print(clean)'`, which displays `'PYTHON'`. The first assignment changes what the name `'clean'` refers to after `'strip'` returns a string; the second assignment does the same after `'upper'` returns a string. The original string object was never edited in place. Now I make a neighboring list bug. I start with `route = ["red", "green"]`, write `'saved = route.append("blue")'`, and print `'saved'`. The output is `'None'`, but printing `'route'` shows `['red', 'green', 'blue']`. That is not a failed append and not a new list hidden in `'saved'`. The list receiver changed in place, while `append` deliberately returned `'None'`. If I wanted a separate list value, I would use `'copy'`, as in `'backup = route.copy()'`, and then check that `'backup'` is a list. These two debugging cases look similar because both use a dot and parentheses, but their method contracts differ. I inspect the receiver, the saved return value, and the post-call state separately. That three-part check usually tells me whether I forgot to save a returned value, incorrectly expected a mutation, or simply called the wrong method.

Before moving on, notice the variable names in this debugging story. `'label'`, `'clean'`, `'route'`, `'saved'`, and `'backup'` are references used to reach objects. They are not labels for permanent

types. A name can be rebound to a returned string or a copied list, while the type of the object it reaches is determined by that current value. This is why printing a result and checking

`'type'` can be more informative than reasoning from a variable's name.

Ask Python what is available

```
signal = "red to green"
print(type(signal).__name__)  # expect: str
print("upper" in dir(signal)) # expect: True
print(signal.replace("green", "blue")) # expect: red to blue
print(signal.find("green"))  # expect: 7
print(signal.startswith("red")) # expect: True
```

You do not need to memorize every method. Python can help you investigate. Type asks what kind of object you have.

You do not need to memorize every method. Python can help you investigate. Type asks what kind of object you have. Dir lists names of attributes and methods available on that object.

It is a menu, not a promise that every name is a method or that every method takes the same arguments. Help gives more focused documentation about a type or a method. A practical

loop is: identify the type, look through the available names, ask for help on a promising method, predict a small result, and run a tiny experiment. Start with a familiar object such as

a string or list, then try the same investigation on something new. If you see a method name in the list, read its help before guessing what it changes or returns. This is a professional

programming habit. The goal is not to know the whole library in your head. The goal is to know how to ask a good question of the language and then test the answer. Documentation

and experiments work together: documentation tells you the contract, while a small run helps you connect that contract to a value you can see.

Recap the evidence

Let's collect the evidence before the final review. An object is a value with a type, state, and available behavior.

Let's collect the evidence before the final review. An object is a value with a type, state, and available behavior. A variable name is a reference to the current object, so the name does not tell us the whole story. A dot performs attribute lookup, and parentheses call a method; without the parentheses we have only referred to the method attribute. For strings, immutability means methods such as 'upper', 'strip', and 'replace' return a string; the original is unchanged. CPython may hand back the same object when nothing changed, so compare values, not identity. 'split' is a useful reminder that the result can be a list, while 'find' gives an integer position and 'startswith' gives a Boolean. For lists, mutability allows in-place changes. 'append' changes the receiver and returns 'None'; 'copy' returns a list. There is no universal "all methods return" or "all methods mutate" rule. The method's documentation is the contract.

When a line surprises you, debug it in a fixed order. Identify the receiver and its type. Read the call carefully, including its parentheses. Save the return value if you need it. Inspect the receiver after the call. Then use 'dir' to discover names, 'help' to read focused documentation, and a tiny run to test one prediction. This routine makes the invisible parts of a method call visible: the object before the dot, the result sent back, and the state that remains afterward. You are not memorizing isolated tricks. You are building a repeatable way to learn an unfamiliar object.

The review is complete. We met values we already use, followed the dot, and compared what string and list methods do.

The review is complete. We met values we already use, followed the dot, and compared what string and list methods do. The reusable habit is this: when Python gives you a thing, ask what type it has, what state it holds, what behavior it offers, what it returns, and whether it changes in place. Strings and lists are not merely containers for syntax practice. They are early examples of a much larger way to organize software. Later, objects may represent files, paths, game entities, or connections. The exact methods will come from their types and documentation, so do not assume that a conceptual example is a real API. For today, keep the contrast sharp. Uppercase on a string returns a string; the original is unchanged. Append on a list changes the list and returns None. Dir and help make unfamiliar behavior discoverable. If you can explain those observations, you already have a working first model of objects. The next time a method surprises you, do not guess that all objects behave like strings or all methods behave like append. Identify the type, read the method's contract, and run the smallest useful check.